01. Lighthouse 프로젝트 구조 및 인덱스

개요

이 문서는 Lighthouse Ethereum 합의 클라이언트의 전체 프로젝트 구조를 체계적으로 분석하고, 각 디렉터리가 어떤 역할군에 속하는지 명확하게 설명합니다. 코드 분석을 시작하기 전에 이 문서를 통해 전체적인 구조를 파악하면 각 컴포넌트의 역할과 상호관계를 쉽게 이해할 수 있습니다.

전체 아키텍처 개요

Lighthouse는 다음과 같은 주요 컴포넌트 그룹으로 구성됩니다:

```
Lighthouse 프로젝트
├── 🏛️ 핵심 실행 바이너리
| ├── lighthouse/ # 메인 CLI 바이너리
| ├── beacon_node/ # 비콘 노드 구현
| └── validator_client/ # 검증자 클라이언트 구현
|
├── 🔧 지원 도구들
| ├── account_manager/ # 계정 관리 도구
| ├── boot_node/ # 부트스트랩 노드
| ├── database_manager/ # 데이터베이스 관리 도구
| ├── lcli/ # 저수준 CLI 도구
| ├── slasher/ # 슬래싱 감지 서비스
```

```
│ └── validator_manager/ # 검증자 관리 도구
│
├── 📚 공통 라이브러리
│ ├── common/ # 공통 유틸리티들
│ ├── consensus/ # 합의 알고리즘 구현
│ └── crypto/ # 암호학 라이브러리들
│
└── 🧪 테스트 및 개발 도구
├── testing/ # 테스트 프레임워크
├── scripts/ # 빌드 및 배포 스크립트
└── book/ # 문서화
```

디렉터리별 상세 분석

🏛️ 핵심 실행 바이너리

1. `lighthouse/` - 메인 CLI 바이너리

역할: 전체 시스템의 진입점이자 통합 CLI 인터페이스

핵심 파일들:

- `src/main.rs` - 메인 함수 및 CLI 파싱
- `environment/` - 런타임 환경 설정
- `Cargo.toml` - 의존성 및 피처 플래그 정의

주요 기능:

- 비콘 노드와 검증자 클라이언트 실행
- 네트워크별 설정 관리 (mainnet, testnet 등)
- 로깅 및 메트릭 설정

2. `beacon_node/` - 비콘 노드 구현

역할: Ethereum 2.0 합의 프로토콜의 핵심 구현체

핵심 서브모듈들:

```
beacon_node/
├── src/lib.rs # ProductionBeaconNode 구조체 정의
├── beacon_chain/ # 🔗 비콘 체인 상태 관리
├── network/ # 🌐 P2P 네트워킹 레이어
├── lighthouse_network/ # 🌐 네트워크 프로토콜 구현
├── execution_layer/ # ⚡ 실행 레이어 통신
├── http_api/ # 🌍 REST API 서버
├── store/ # 💾 데이터 저장소
├── operation_pool/ # 📦 오퍼레이션 풀 관리
├── beacon_processor/ # ⚙️ 블록/증명 처리
├── genesis/ # 🌱 제네시스 상태 생성
├── http_metrics/ # 📊 메트릭 수집
└── timer/ # ⏰ 타이밍 서비스
```

주요 진입점:

- `ProductionBeaconNode::new()` - 비콘 노드 초기화

- `ClientBuilder` - 빌더 패턴을 통한 설정

3. `validator_client/` - 검증자 클라이언트 구현

역할: 검증자 역할 수행 및 듀티 관리

핵심 서브모듈들:

```
validator_client/
├── src/lib.rs # ProductionValidatorClient 구조체
├── validator_services/ # 🎯 핵심 검증자 서비스들
│ ├── duties_service.rs # 듀티 관리
│ ├── attestation_service.rs # 증명 생성/제출
│ ├── block_service.rs # 블록 제안
│ └── sync_committee_service.rs # 동기화 위원회
├── lighthouse_validator_store/ # 🔐 검증자 키 저장소
├── slashing_protection/ # 🛡️ 슬래싱 보호
├── beacon_node_fallback/ # 🔄 비콘 노드 장애조치
├── doppelganger_service/ # 👥 중복 검증자 감지
├── signing_method/ # ✍️ 서명 방법 관리
├── http_api/ # 🌐 검증자 API
└── validator_metrics/ # 📊 검증자 메트릭
```

주요 진입점:

- `ProductionValidatorClient::new()` - 검증자 클라이언트 초기화
- 각 서비스의 `Builder` 패턴

🔧 지원 도구들

1. `account_manager/` - 계정 관리 도구

역할: 검증자 계정 생성, 가져오기, 관리

주요 기능:

- 키스토어 생성 및 관리
- 검증자 등록
- 계정 백업 및 복원

2. `slasher/` - 슬래싱 감지 서비스

역할: 악의적인 검증자 행동 감지 및 신고

핵심 컴포넌트:

- `service/` - 슬래싱 감지 서비스
- 데이터베이스 백엔드 (LMDB, MDBX, redb)

3. `boot_node/` - 부트스트랩 노드

역할: 새로운 노드들의 네트워크 진입점 제공

📚 공통 라이브러리

1. `common/` - 공통 유틸리티 라이브러리

주요 서브모듈들:

```
common/
├── eth2/ # 🌐 Ethereum 2.0 API 클라이언트
├── types/ # 📋 공통 데이터 타입들
├── slot_clock/ # ⏰ 슬롯 시간 관리
```

```
├── task_executor/ # 🔄 비동기 작업 실행
├── metrics/ # 📊 메트릭 수집 프레임워크
├── logging/ # 📝 로깅 시스템
├── directory/ # 📁 디렉터리 관리
├── account_utils/ # 👤 계정 유틸리티
├── validator_dir/ # 📂 검증자 디렉터리 관리
├── eth2_config/ # ⚙️ Ethereum 2.0 설정
├── eth2_network_config/ # 🌐 네트워크 설정
└── ... (기타 유틸리티들)
```

2. `consensus/` - 합의 알고리즘 구현

핵심 서브모듈들:

```
consensus/
├── types/ # 📋 합의 관련 데이터 타입
├── state_processing/ # 🔄 상태 전이 함수들
├── fork_choice/ # 🍴 포크 선택 알고리즘
├── proto_array/ # 🌳 ProtoArray 구현
├── safe_arith/ # 🔢 안전한 산술 연산
├── merkle_proof/ # 🌲 머클 증명
└── ... (기타 합의 관련 모듈들)
```

주요 진입점:

- `BeaconState` - 비콘 체인 상태
- `BeaconBlock` - 블록 구조
- `process_slots()`, `process_block()` - 상태 전이 함수들

3. `crypto/` - 암호학 라이브러리

핵심 서브모듈들:

```
crypto/
├── bls/ # 🔐 BLS 서명 및 검증
├── kzg/ # 📊 KZG 커밋먼트 (EIP-4844)
├── eth2_keystore/ # 🔑 키스토어 관리
├── eth2_key_derivation/ # 🔑 키 파생
└── eth2_wallet/ # 👛 지갑 관리
```

주요 진입점:

- `Signature::verify()` - BLS 서명 검증
- `Keystore::decrypt()` - 키스토어 복호화

🧪 테스트 및 개발 도구

1. `testing/` - 테스트 프레임워크

주요 서브모듈들:

```
testing/
├── ef_tests/ # 🧪 Ethereum Foundation 공식 테스트
├── simulator/ # 🎮 네트워크 시뮬레이터
```

```
├── node_test_rig/ # 🔧 노드 테스트 도구
├── validator_test_rig/ # ✅ 검증자 테스트 도구
├── execution_engine_integration/ # ⚡ 실행 엔진 통합 테스트
└── state_transition_vectors/ # 📊 상태 전이 테스트 벡터
```

2. `scripts/` - 빌드 및 배포 스크립트

주요 스크립트들:

- `local_testnet/` - 로컬 테스트넷 구성
- `ci/` - CI/CD 스크립트들

코드 분석 시 참고사항

주요 진입점들

1. **메인 바이너리 시작점**:

```rust
// lighthouse/src/main.rs
fn main() -> Result<(), String>
```

2. **비콘 노드 초기화**:

```rust
// beacon_node/src/lib.rs
impl<E: EthSpec> ProductionBeaconNode<E> {
pub async fn new(context: RuntimeContext<E>, client_config: ClientConfig) ->
}
```

3. **검증자 클라이언트 초기화**:

```
// validator_client/src/lib.rs

impl<E: EthSpec> ProductionValidatorClient<E> {

pub  async  fn  new(context: RuntimeContext<E>, config: Config) -> Result<Self,

}
```

핵심 데이터 구조들

1. **비콘 상태**: `consensus/types/src/beacon_state.rs`
2. **블록 구조**: `consensus/types/src/beacon_block.rs`
3. **증명**: `consensus/types/src/attestation.rs`
4. **검증자 정보**: `consensus/types/src/validator.rs`

중요한 트레이트들

1. **EthSpec**: Ethereum 스펙 정의 (`consensus/types/src/eth_spec.rs`)
2. **SlotClock**: 시간 관리 (`common/slot_clock/src/lib.rs`)
3. **ValidatorStore**: 검증자 저장소 인터페이스

학습 순서 권장사항

이 문서 시리즈를 효과적으로 학습하기 위한 권장 순서:

1. **기초 이해** (문서 01-03)

- 프로젝트 구조 (현재 문서)
- 전체 아키텍처 개요

- 핵심 데이터 타입들

2. **비콘 노드 심화** (문서 04-08)

- Beacon Node 구현
- Beacon Chain 상태 관리
- Fork Choice 알고리즘
- 네트워킹 및 P2P

3. **검증자 클라이언트 심화** (문서 09-14)

- Validator Client 아키텍처
- 각종 서비스들 (Duties, Attestation, Block, Sync Committee)
- 서명 및 보안

4. **암호학 및 고급 주제** (문서 15-21)

- BLS 암호학
- 키 관리
- KZG 커밋먼트
- 실행 레이어 통신

5. **실용적 주제** (문서 22-25)

- 테스트 및 시뮬레이션
- 성능 최적화
- 설정 관리
- 통합 가이드

다음 문서 미리보기

02. Lighthouse 전체 아키텍처 개요: 각 컴포넌트들이 어떻게 상호작용하는지, 데이터 흐름과 제어 흐름을 중심으로 전체 시스템의 동작 원리를 설명합니다.

이 문서를 통해 Lighthouse의 전체적인 구조를 파악했다면, 이제 각 컴포넌트의 상세한 구현을 분석할 준비가 되었습니다!